

Project Hub — folder guide

What each folder is for, and the order they fill in.

● Product ● Developer ● Both ● Standard (rules)

PART 1 · WHAT EACH FOLDER IS FOR

● <code>product/</code>	Product context — prd.md , roadmap.md . Written by PM/PO in Grooming.
● <code>product/knowledge-base/</code>	Discovery inputs — analytics, feedback, competitors, support, RFPs, trends. Read by the Analyst.
● <code>technical/</code>	System architecture.md + decisions adr/ . Written by the Architect.
● <code>technical/knowledge-base/</code>	Discovery inputs — static analysis, deps, crashes, perf. Read by tech-discovery agents.
● <code>epics/EPIC-x/</code>	One folder = one epic's full context : <code>epic.md</code> → <code>ux/</code> → <code>architecture.md</code> → <code>stories/</code> .
● <code>epics/EPIC-x/stories/</code>	The unit that crosses into Jira . Each story carries <code>jiraKey</code> , <code>repos[]</code> , echoed <code>status</code> .
● <code>repos/registry.yaml</code>	The multi-repo map. A story's <code>repos: []</code> resolves here → engine clones those repos.
● <code>process/</code>	<code>workflow.yaml</code> (phases→agent→gate), <code>agents/</code> (personas), <code>templates/</code> . The engine reads these.
● <code>CONVENTIONS.md</code>	Frontmatter schemas + the two one-way sync rules. Keeps Jira & Hub aligned.

PART 2 · THE ORDER THEY FILL

ZONE 1 · in the Hub (no Jira yet)

- 1 Discovery** · Analyst + Architect
writes `epic.md`, updates `technical/architecture.md` + an ADR
- 2 Design** · UX Expert
writes `ux/design-spec.md`
- 3 Grooming** · PM + PO
updates `prd.md`, writes `stories/STORY-*.md` with `repos[]`
↳ **handoff** — stories mirrored to Jira, each gets a `jiraKey`

ZONE 2 · Jira drives (agents still write the Hub + code repos)

- 4 **Solution** · Architect + SM
writes `epics/EPIC-x/architecture.md`; SM sets points
- 5 **Implementation** · Dev → Review → QA
clones the story's repos → writes **code** (1 PR each); status echoes from Jira
- 6 **Release** · Deploy + Verify
deploys, verifies; story → **DONE**. Context stays in the Hub.

`process/` and `CONVENTIONS.md` are never written during a run — they're the rules the engine reads throughout.